

# Un Vision Transformer desde cero en C con aceleración por GPU

## ¿Compensa frente a una arquitectura definida en Python?

Ezequiel González Tortajada

26 de septiembre de 2026

### Resumen

Este trabajo implementa desde cero, en C99 y sin librerías de aprendizaje profundo, un *Vision Transformer* completo: tensores, retropropagación de cada operación, optimizador AdamW, aumentación de datos, evaluación con *ensemble* y *test-time augmentation*, y dos backends de GPU, uno para Metal (Apple Silicon) y otro para CUDA (NVIDIA), que ejecutan el paso de entrenamiento completo en el dispositivo. Sobre el conjunto de datos OrganSMNIST, con cortes de tomografía abdominal de  $28 \times 28$  píxeles clasificados en 11 órganos, un modelo de 0.80 millones de parámetros entrenado sin pre-entrenamiento alcanza un 75.9 % de precisión en test, y un *ensemble* de dos modelos con TTA un 78.6 %. El modelo individual queda 2.8 puntos por debajo de una ResNet-50 pre-entrenada y ajustada a la misma resolución en PyTorch (78.7 %, 23.5 millones de parámetros), a la que el *ensemble* iguala; ambos quedan por debajo del mejor modelo de referencia, un ViT-small pre-entrenado a  $64 \times 64$  (82.2 %). La comparación muestra que el rendimiento computacional se puede alcanzar con un esfuerzo acotado, pero que el coste real de construir desde cero está en la verificación: el error más grave del proyecto, un gradiente nulo en la atención, redujo la precisión al 51.6 % sin producir ningún fallo visible. Se concluye que, para obtener la mejor precisión en un *benchmark*, una arquitectura definida en Python es claramente preferible; la implementación propia compensa cuando el objetivo es comprender el modelo, desplegarlo sin dependencias o controlar los kernels.

## Índice

|                                              |          |
|----------------------------------------------|----------|
| <b>1. Introducción</b>                       | <b>3</b> |
| <b>2. Datos y protocolo</b>                  | <b>3</b> |
| 2.1. OrganSMNIST . . . . .                   | 3        |
| 2.2. Protocolo . . . . .                     | 3        |
| 2.3. Dos versiones de las imágenes . . . . . | 4        |
| <b>3. Implementación</b>                     | <b>4</b> |
| 3.1. Modelo . . . . .                        | 4        |
| 3.2. Entrenamiento . . . . .                 | 4        |
| 3.3. Verificación . . . . .                  | 5        |
| 3.4. Backends de GPU . . . . .               | 5        |
| <b>4. Rendimiento computacional</b>          | <b>6</b> |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>5. Resultados</b>                                        | <b>7</b>  |
| 5.1. Resultado principal . . . . .                          | 7         |
| 5.2. Un error silencioso . . . . .                          | 7         |
| 5.3. Tamaño de la arquitectura . . . . .                    | 7         |
| 5.4. El límite de la lateralidad . . . . .                  | 8         |
| <b>6. Comparación con arquitecturas definidas en Python</b> | <b>8</b>  |
| 6.1. Precisión y coste . . . . .                            | 8         |
| 6.2. Esfuerzo de desarrollo . . . . .                       | 9         |
| <b>7. Conclusiones</b>                                      | <b>10</b> |
| 7.1. ¿Compensa? . . . . .                                   | 10        |
| 7.2. Recomendación . . . . .                                | 10        |
| 7.3. Trabajo futuro . . . . .                               | 10        |

## 1. Introducción

Entrenar un *Vision Transformer* [2] con PyTorch [7] y la librería `timm` [8] requiere unas decenas de líneas: la arquitectura, la diferenciación automática, el optimizador y la ejecución en GPU vienen resueltos, y se dispone de pesos pre-entrenados sobre millones de imágenes. Este trabajo recorre el camino opuesto: construir el mismo tipo de modelo desde cero en C, sin dependencias, incluyendo su ejecución en GPU.

La pregunta que guía la memoria es si ese esfuerzo compensa. Para responderla con datos, se entrena en el mismo portátil y con el mismo protocolo una referencia en Python: 13 modelos en PyTorch y scikit-learn, desde modelos clásicos hasta redes pre-entrenadas con *fine-tuning*, entre ellas un ViT-small.

Las contribuciones son:

- Un ViT entrenable de extremo a extremo en C99, con retropropagación verificada por diferencias finitas.
- Un diseño de backend de dispositivo con memoria compartida y una sola sincronización por paso de entrenamiento, implementado para Metal y para CUDA.
- Kernels de atención en tres variantes, incluida una por bloques de tipo FlashAttention [6] para secuencias largas.
- Una comparación cuantitativa de precisión, coste computacional y esfuerzo de desarrollo frente a modelos definidos en Python.

## 2. Datos y protocolo

### 2.1. OrganSMNIST

OrganSMNIST pertenece a la colección MedMNIST v2 [1]. Contiene cortes sagitales de tomografía computarizada abdominal de  $28 \times 28$  píxeles en escala de grises, etiquetados en 11 órganos, con una partición oficial de 13 932 imágenes de entrenamiento, 2 452 de validación y 8 827 de test. La tabla 1 recoge los resultados publicados por los autores de la colección.

| método               | AUC   | precisión |
|----------------------|-------|-----------|
| ResNet-18 (28)       | 0.972 | 78.2 %    |
| ResNet-18 (224)      | 0.974 | 77.8 %    |
| ResNet-50 (28)       | 0.972 | 77.0 %    |
| ResNet-50 (224)      | 0.975 | 78.5 %    |
| auto-sklearn         | 0.945 | 67.2 %    |
| AutoKeras            | 0.974 | 81.3 %    |
| Google AutoML Vision | 0.964 | 74.9 %    |

**Tabla 1:** Resultados oficiales de MedMNIST v2 sobre el test de OrganSMNIST.

### 2.2. Protocolo

Todos los modelos, propios y de referencia, siguen el mismo protocolo, lo que hace comparables las cifras:

- *Particiones.* Entrenamiento ajusta los pesos; validación selecciona el checkpoint, la arquitectura y la configuración de evaluación; test se consulta solo para informar.
- *Preproceso.* Los píxeles se normalizan con la media y la desviación típica del conjunto de entrenamiento (0.495 y 0.283).
- *Métricas.* Precisión, precisión equilibrada (media del *recall* por clase), macro-F1 y matriz de confusión.

## 2.3. Dos versiones de las imágenes

OrganSMNIST circula en dos formatos: el fichero oficial, con las imágenes ya reducidas a  $28 \times 28$  por los autores de la colección, y copias en repositorios de terceros con las imágenes en alta resolución. Parte del desarrollo se hizo con estas últimas, reducidas a  $28 \times 28$  con la interpolación de la librería Pillow. Las particiones y las etiquetas coinciden, pero los píxeles no: la desviación típica del conjunto de entrenamiento es 0.262 en esa versión y 0.283 en la oficial.

La diferencia no es inocua. Con la misma configuración, la versión reducida con Pillow alcanza un 78.4 y un 78.6 % en test en dos ejecuciones, y la oficial un 75.9 y un 76.2 % con dos semillas. Todas las cifras principales de esta memoria usan la versión oficial, que es la de los modelos de referencia y la de la tabla 1. Solo el estudio de arquitecturas de la sección 5.3, que compara variantes entre sí, usa la otra versión.

## 3. Implementación

### 3.1. Modelo

El modelo es un ViT con normalización previa (*Pre-LN*) [3]. Cada imagen se divide en parches no solapados de  $P \times P$  píxeles, que se proyectan a dimensión  $d$ ; se antepone un token [CLS] y se suman posiciones aprendibles. Cada bloque aplica

$$\begin{aligned} h &= x + \text{MHA}(\text{LN}(x)), \\ y &= h + W_2 \text{GELU}(W_1 \text{LN}(h)), \end{aligned}$$

con atención multi-cabeza bidireccional y un MLP de expansión 4. Tras el último bloque, una LayerNorm y una capa lineal sobre [CLS] producen los *logits*. La configuración principal ( $P = 4$ ,  $d = 128$ , 8 cabezas, 4 bloques) tiene 50 tokens por imagen y 803 467 parámetros.

### 3.2. Entrenamiento

No hay diferenciación automática: cada operación tiene un *forward* que guarda en una caché las activaciones necesarias y un *backward* escrito a mano que acumula gradientes. El entrenamiento usa AdamW [4] con *weight decay* desacoplado, *clipping* global por norma, *label smoothing* [5], *linear warmup* y *cosine decay*. La aumentación de datos se limita a transformaciones que conservan la orientación anatómica (traslación, rotación, escala, brillo, contraste, ruido y *random erasing*). Se excluyen los volteos, que invierten la orientación de la imagen, siempre la misma en los datos.

La evaluación admite *ensembles* de checkpoints, que promedian probabilidades y pueden mezclar arquitecturas, y *test-time augmentation* con 5 o 9 vistas desplazadas un píxel.

### 3.3. Verificación

Como no hay diferenciación automática, cada gradiente es una fuente potencial de error silencioso. La verificación se apoya en dos mecanismos:

- *Diferencias finitas.* Se comparan los gradientes analíticos con  $\frac{f(\theta+\epsilon)-f(\theta-\epsilon)}{2\epsilon}$  en las operaciones y en el bloque completo.
- *CPU como referencia.* La implementación en C es la referencia numérica. Un test entrena el mismo modelo en CPU y en GPU y exige que coincidan la pérdida, los gradientes de todos los parámetros y la trayectoria de entrenamiento, en tres configuraciones que cubren las tres variantes del kernel de atención.

### 3.4. Backends de GPU

Ambos backends siguen el mismo diseño, resumido en la tabla 2:

1. *Memoria compartida.* Al activar el backend, cada tensor se reserva en memoria accesible desde CPU y GPU. La CPU solo accede a ella tras sincronizar.
2. *Per-operation dispatch.* Cada operación de la librería consulta el backend instalado; si todos sus tensores residen en el dispositivo, encola el kernel sin esperar, y si no, sincroniza y ejecuta la versión en C.
3. *Una sincronización por paso.* El paso de entrenamiento completo, desde la extracción de parches hasta AdamW, se encola entero y solo se sincroniza al leer la pérdida.

|                            | Metal                            | CUDA               |
|----------------------------|----------------------------------|--------------------|
| memoria compartida         | MTLBuffer compartido             | cudaMallocManaged  |
| multiplicación de matrices | MPSMatrixMultiplication          | cuBLAS cublasSgemv |
| cola de trabajo            | <i>command buffer</i> pendiente  | cudaStream_t       |
| reducciones                | SIMD-groups de 32 hilos          | <i>warps</i>       |
| parámetros agrupados       | tabla de <code>gpuAddress</code> | tabla de punteros  |

**Tabla 2:** Correspondencia entre los dos backends.

La atención elige entre tres kernels. Si la secuencia completa cabe en memoria compartida ( $P = 4, 50$  tokens), un grupo de 256 hilos por par (muestra, cabeza) calcula todas las fases en paralelo; el *backward* usa  $D_i = dO_i \cdot O_i$  para no almacenar  $dP$  [6]. Si no cabe ( $P = 2, 197$  tokens), un kernel por bloques de 32 consultas recorre las claves por bloques con *softmax* en línea. Para cabezas de dimensión mayor que 64 queda un kernel genérico.

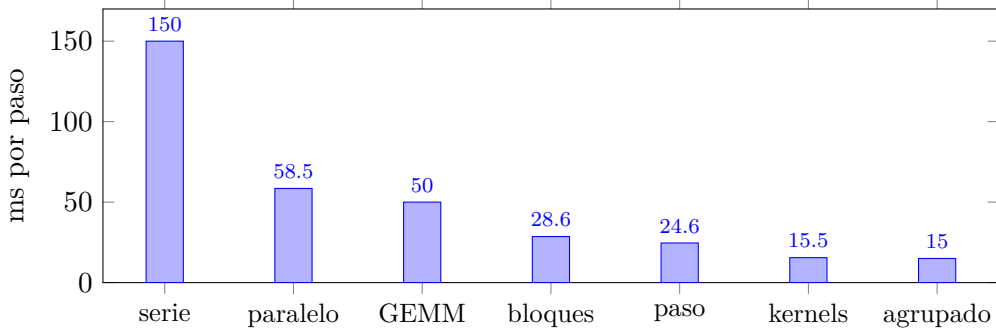
El backend CUDA replica el de Metal kernel a kernel y se ha validado en una GPU NVIDIA L40S. El test de comparación con la CPU pasa en las tres configuraciones de la atención. Además, con los mismos datos y la misma semilla, las dos primeras épocas de entrenamiento (870 pasos) producen en la L40S y en el M3 la misma pérdida y la misma precisión de validación hasta la sexta cifra decimal: dos GPU de fabricantes distintos, con kernels escritos por separado, reproducen el mismo entrenamiento.

## 4. Rendimiento computacional

Todas las medidas se toman en un MacBook con chip M3 (8 núcleos de CPU y GPU integrada), con *batch* de 32 y la configuración principal. La tabla 3 resume la evolución del tiempo por paso de entrenamiento.

| versión                                      | ms por paso | muestras por segundo |
|----------------------------------------------|-------------|----------------------|
| CPU, multiplicación de matrices ingenua      | 3 967       | 8                    |
| CPU en serie + multiplicaciones en GPU (MPS) | 150         | 213                  |
| + paralelismo en CPU (8 núcleos)             | 58.5        | 547                  |
| + multiplicaciones sin transponer en CPU     | 50.0        | 640                  |
| + operaciones de los bloques en GPU          | 28.6        | 1 121                |
| + paso completo en GPU, una sincronización   | 24.6        | 1 300                |
| + kernels optimizados                        | 15.5        | 2 065                |
| + <i>clipping</i> y AdamW agrupados          | 15.0        | 2 133                |

**Tabla 3:** Tiempo por paso de entrenamiento ( $P = 4$ ,  $d = 128$ , 4 bloques).



**Figura 1:** Evolución del tiempo por paso a partir de la primera versión con GPU.

Tres observaciones se desprenden del perfilado:

- *La GPU solo acelera si el trabajo reside en ella.* Enviar a la GPU únicamente las multiplicaciones de matrices, copiando operandos y esperando cada una, dejaba la GPU al 18 % de uso: el tiempo se iba en la CPU y en las sincronizaciones. El salto grande llega al mover todas las operaciones y encolar el paso completo.
- *Los kernels ingenuos desperdician la GPU.* Las reducciones por columnas (gradientes de sesgos,  $\gamma$  y  $\beta$ ) con un hilo por columna recorrían 1 600 filas en serie. Reescritas con bloques de  $32 \times 32$  hilos pasaron de 4.1 a 0.3 ms por paso; la LayerNorm, con un SIMD-group por fila, de 3.3 a 0.3 ms.
- *Un primer intento de optimización empeoró el resultado.* Cargar la atención en memoria compartida con un hilo por consulta dejaba 50 de 256 hilos activos y multiplicó por 2.7 el tiempo del *backward*; repartir cada fase entre todos los hilos lo redujo a 0.65 ms por llamada.

Con parches de  $2 \times 2$ , la secuencia pasa de 50 a 197 tokens. El kernel por bloques reduce el paso de 152 a 101 ms, pero cada época sigue costando unos 70 s frente a los 8 s de la configuración principal.

## 5. Resultados

### 5.1. Resultado principal

La tabla 4 recoge los resultados de la configuración principal con la versión oficial de los datos. Se entrenan dos modelos con semillas distintas y se combinan en un *ensemble*; el número de vistas de TTA se elige por la precisión de validación.

| modelo                            | validación    | test          | equilibrada  | macro-F1     |
|-----------------------------------|---------------|---------------|--------------|--------------|
| semilla 42                        | 88.1 %        | 75.9 %        | 0.718        | 0.721        |
| semilla 7                         | 87.2 %        | 76.2 %        | 0.724        | 0.719        |
| <i>ensemble</i> , sin TTA         | 89.0 %        | 77.4 %        | 0.735        | 0.742        |
| <i>ensemble</i> , TTA de 5 vistas | 89.8 %        | 78.4 %        | 0.745        | 0.752        |
| <i>ensemble</i> , TTA de 9 vistas | <b>90.3 %</b> | <b>78.6 %</b> | <b>0.745</b> | <b>0.754</b> |

**Tabla 4:** Resultados de la configuración principal ( $P = 4$ ,  $d = 128$ , 4 bloques) sobre la versión oficial de los datos.

Las dos semillas difieren en 0.3 puntos en test. Promediar ambos modelos aporta 1.4 puntos, y la TTA otros 1.2.

### 5.2. Un error silencioso

Durante el desarrollo, con la versión de los datos reducida con Pillow, el *backward* de la atención enrutaba el gradiente de la salida en sentido contrario, de modo que las proyecciones de consultas, claves y valores nunca recibían gradiente y conservaban su inicialización aleatoria. El programa entrenaba, la pérdida bajaba y la precisión alcanzaba un 51.6 %: un resultado plausible para un modelo pequeño. Solo una comprobación por diferencias finitas sobre los pesos de la atención, que el test existente no incluía, reveló que el gradiente era exactamente cero. Tras corregirlo, junto con la normalización de la entrada y una inicialización aleatoria real, la precisión en validación llegó al 81 % en cinco épocas.

### 5.3. Tamaño de la arquitectura

La tabla 5 compara variantes entrenadas con los mismos hiperparámetros sobre la versión de los datos reducida con Pillow (sección 2.3). Sirve para comparar las variantes entre sí, no con el resto de cifras de la memoria. Se seleccionan por la pérdida de validación.

| configuración                   | parámetros | pérdida val. | precisión val. | test   |
|---------------------------------|------------|--------------|----------------|--------|
| $P = 4$ , $d = 128$ , 4 bloques | 0.80 M     | <b>0.311</b> | <b>91.0 %</b>  | 78.4 % |
| $P = 4$ , $d = 192$ , 6 bloques | 2.68 M     | 0.363        | 89.0 %         | 77.4 % |
| $P = 2$ , $d = 128$ , 4 bloques | 0.82 M     | 0.347        | 89.1 %         | 77.8 % |

**Tabla 5:** Variantes de arquitectura, con la versión de los datos reducida con Pillow.

Ninguna variante mejora la configuración de partida. Con 13932 imágenes y sin pre-entrenamiento, aumentar la profundidad, la anchura o la resolución de los parches no aporta: el cuello de botella no está en la capacidad del modelo.

## 5.4. El límite de la lateralidad

Casi un tercio de los errores del mejor modelo individual (el 32 %) se concentra en dos pares: fémur izquierdo y derecho, y riñón izquierdo y derecho. Para medir cuánta información contienen las imágenes sobre la lateralidad se clasifica cada par con el criterio más simple posible, la distancia a la imagen media de cada clase:

| par   | media más cercana (val.) | media más cercana (test) | ViT, dentro del par (test) |
|-------|--------------------------|--------------------------|----------------------------|
| fémur | 62.3 %                   | 51.5 %                   | 50.4 %                     |
| riñón | 75.6 %                   | 61.8 %                   | 72.4 %                     |

**Tabla 6:** Acierto al separar el órgano izquierdo del derecho en cada par.

En test, los fémures se separan al azar con cualquier criterio, y el ViT no lo mejora. En los riñones queda algo de señal, que el ViT aprovecha. La validación vuelve a ser más optimista que el test, como ocurre con todos los modelos de esta memoria. La explicación es anatómica: en un corte sagital, la lateralidad depende de la profundidad a la que se tomó el corte, no del contenido de la imagen.

## 6. Comparación con arquitecturas definidas en Python

### 6.1. Precisión y coste

La tabla 7 reúne los principales modelos de referencia en PyTorch y los de este trabajo. Todos se entrenaron en el mismo portátil, PyTorch con su backend MPS y el ViT en C con el backend Metal propio.

| modelo                   | origen  | pre-entr. | resol. | parámetros | test          | equil.       | tiempo  |
|--------------------------|---------|-----------|--------|------------|---------------|--------------|---------|
| perceptrón multicapa     | PyTorch | no        | 28     | 0.67 M     | 59.1 %        | 0.558        | 28 s    |
| LeNet                    | PyTorch | no        | 28     | 0.06 M     | 69.4 %        | 0.644        | 40 s    |
| ResNet-50, completo      | PyTorch | sí        | 28     | 23.5 M     | 78.7 %        | 0.746        | 407 s   |
| ResNet-50                | PyTorch | sí        | 64     | 23.5 M     | 80.9 %        | 0.763        | 744 s   |
| ViT-small                | PyTorch | sí        | 64     | 21.4 M     | <b>82.2 %</b> | <b>0.784</b> | 1 065 s |
| ViT desde cero           | C       | no        | 28     | 0.80 M     | 75.9 %        | 0.718        | 566 s   |
| <i>ensemble</i> ×2 + TTA | C       | no        | 28     | 1.61 M     | 78.6 %        | 0.745        | 1 208 s |

**Tabla 7:** Modelos de referencia en PyTorch y modelos de este trabajo sobre el test de OrganSMNIST. El tiempo es el de entrenamiento: 12 épocas para los modelos pre-entrenados, 30 para las redes desde cero de PyTorch y 50 para el ViT en C.

Cuatro lecturas:

- *El modelo individual queda algo por debajo; el ensemble, a la par.* Entrenado desde cero a  $28 \times 28$ , el ViT en C queda 2.8 puntos por debajo de la ResNet-50 pre-entrenada a la misma resolución y 2.3 puntos por debajo de la ResNet-18 oficial (tabla 1), con 29 y 14 veces menos parámetros. El *ensemble* con TTA alcanza el 78.6 %, al nivel de ambas. Frente a la red de PyTorch entrenada desde cero, LeNet, el modelo individual mejora 6.5 puntos.
- *La diferencia con el mejor modelo la marca el ecosistema, no el lenguaje.* Los 6.3 puntos que separan el ViT en C del ViT-small de PyTorch (3.6 con el *ensemble*) se deben sobre

todo a dos cosas que el lenguaje no aporta por sí mismo: el pre-entrenamiento sobre ImageNet-21k y la resolución de  $64 \times 64$ . La sección anterior muestra que ampliar el modelo sin pre-entrenar no cierra esa distancia.

- *El coste computacional es comparable.* Tras la optimización, una época cuesta entre 8 y 13 s según la ejecución, y las 50 épocas, unos 10 minutos. Los tiempos no son directamente comparables, porque cambian el número de épocas, la resolución y el tamaño de los modelos, pero en el mismo portátil el ViT en C tarda algo más que el *fine-tuning* de la ResNet-50 a  $28 \times 28$  y la mitad que el del ViT-small.
- *Las diferencias pequeñas no son significativas.* En la referencia, dos ejecuciones de la misma ResNet-50 difirieron en 1.1 puntos; aquí, dos semillas del ViT difieren en 0.3. Diferencias por debajo de un punto no permiten ordenar modelos.

## 6.2. Esfuerzo de desarrollo

La tabla 8 compara el tamaño del código, sin contar líneas en blanco.

| componente                                                       | líneas |
|------------------------------------------------------------------|--------|
| núcleo en C (tensores, operaciones, ViT, aumentación, datos)     | 3 940  |
| backend Metal (Objective-C++ y kernels Metal)                    | 2 673  |
| backend CUDA                                                     | 1 980  |
| programa de entrenamiento y evaluación                           | 1 021  |
| tests                                                            | 1 216  |
| total de la implementación propia (un ViT y dos backends de GPU) | 10 830 |
| referencia en PyTorch y scikit-learn (13 modelos)                | 899    |

**Tabla 8:** Tamaño del código de cada enfoque.

La referencia entrena 13 modelos, de la regresión logística al ViT-small, con un orden de magnitud menos de código que la implementación propia de un único modelo. Pero la diferencia importante no es de volumen, sino de tipo de error. Los fallos encontrados durante el desarrollo fueron:

- el gradiente nulo en la atención, que no producía ningún fallo visible y dejaba el modelo en un 51.6 % de precisión;
- una inicialización pseudoaleatoria con estructura de diente de sierra, que no era aleatoria;
- un planificador coseno que llegaba a una tasa de aprendizaje exacta de cero en la última época y abortaba el entrenamiento antes de evaluar;
- un objeto liberado por la gestión automática de memoria de Objective-C al reordenar una caché, que cerraba el programa;
- una variable ocultada en el backend CUDA, detectada por análisis estático antes de ejecutarlo;
- dos versiones de las imágenes con píxeles distintos, que desplazaban los resultados 2.5 puntos sin que ningún test pudiera detectarlo.

Con PyTorch, la diferenciación automática elimina el primero, la inicialización y los planificadores de la librería eliminan el segundo y el tercero, y la gestión de memoria del *framework*

elimina el cuarto. Los dos últimos no dependen del lenguaje: pueden ocurrir con cualquier herramienta. Los fallos de un *framework* maduro suelen ser visibles; los de una implementación propia pueden producir un modelo que entrena, converge y da resultados plausibles siendo incorrecto. Por eso una implementación propia exige infraestructura de verificación: diferencias finitas, una referencia numérica independiente y tests que comparen dispositivos.

## 7. Conclusiones

### 7.1. ¿Compensa?

Depende del objetivo.

*Para obtener la mejor precisión en un problema concreto, no compensa.* Una arquitectura definida en Python ofrece, con un orden de magnitud menos de código, modelos pre-entrenados que aportan hasta 6 puntos sobre este problema, diferenciación automática que elimina la clase de error más grave encontrada, y un ciclo de experimentación mucho más corto: la referencia cubre 13 modelos con menos de mil líneas.

*Compensa cuando el objetivo es otro:*

- *Comprender el modelo.* Escribir cada retropropagación obliga a entender dónde fluye cada gradiente; el error de la atención solo se explica entendiendo la operación.
- *Desplegar sin dependencias.* El modelo final ocupa 3.2 MB de pesos, 0.8 millones de parámetros, y se ejecuta con una librería en C99 sin intérprete de Python ni *runtime* de aprendizaje profundo.
- *Controlar el cómputo.* Decidir cuándo se sincroniza la GPU, cómo se reparten los hilos de un kernel o cómo se agrupan los parámetros permitió reducir el paso de entrenamiento de 150 a 15 ms, y ese conocimiento se traslada a cualquier *framework*.

El resultado más relevante es que la parte que parecía más difícil, el rendimiento en GPU, resultó abordable y medible paso a paso. Lo costoso fue la corrección: asegurarse de que cada gradiente es el correcto.

### 7.2. Recomendación

Un enfoque mixto aprovecha lo mejor de ambos: prototipar y seleccionar el modelo en Python, con pesos pre-entrenados; usar el modelo de Python como referencia numérica; y portar a C solo la inferencia, o el entrenamiento cuando el despliegue lo requiera, verificando capa a capa contra esa referencia.

### 7.3. Trabajo futuro

- Medir el rendimiento del backend CUDA en GPU dedicadas y compararlo con PyTorch en el mismo hardware.
- Cargar pesos pre-entrenados de un ViT de PyTorch en la implementación en C, para separar el efecto del pre-entrenamiento del de la implementación.
- Entrenar a  $64 \times 64$  con la atención por bloques.
- Precisión reducida (fp16 o bf16) en las multiplicaciones de matrices.

## Referencias

- [1] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister y B. Ni. MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10:41, 2023. <https://doi.org/10.1038/s41597-022-01721-8>.
- [2] A. Dosovitskiy *et al.* An image is worth 16x16 words: Transformers for image recognition at scale. En *ICLR*, 2021. <https://arxiv.org/abs/2010.11929>.
- [3] R. Xiong *et al.* On layer normalization in the Transformer architecture. En *ICML*, 2020. <https://arxiv.org/abs/2002.04745>.
- [4] I. Loshchilov y F. Hutter. Decoupled weight decay regularization. En *ICLR*, 2019. <https://arxiv.org/abs/1711.05101>.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens y Z. Wojna. Rethinking the Inception architecture for computer vision. En *CVPR*, 2016. <https://doi.org/10.1109/CVPR.2016.308>.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. En *NeurIPS*, 2022. <https://arxiv.org/abs/2205.14135>.
- [7] A. Paszke *et al.* PyTorch: An imperative style, high-performance deep learning library. En *NeurIPS*, 2019. <https://arxiv.org/abs/1912.01703>.
- [8] R. Wightman. PyTorch Image Models, 2019. <https://github.com/huggingface/pytorch-image-models>.